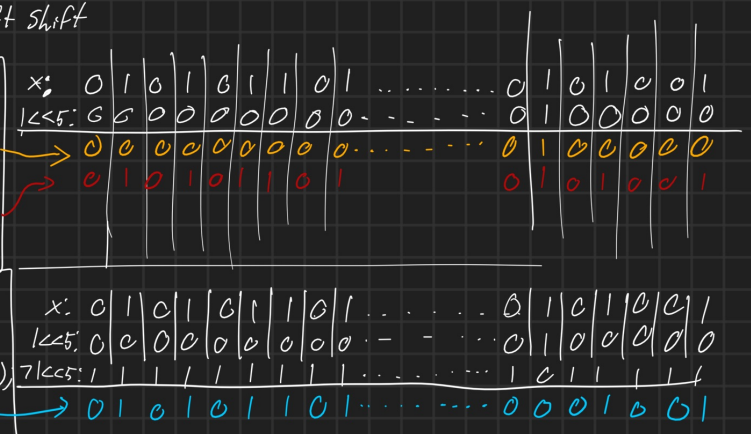
Today:
- Bitwise OPS
  - & | ^ ~ << >>
- logical ops
  - && || ! short circuit
- Assignment, ++x, x++
- Storage Classes

Bitwise operations: Left Shift

```
int f(int x){
    return x & (1<<5);
}
```

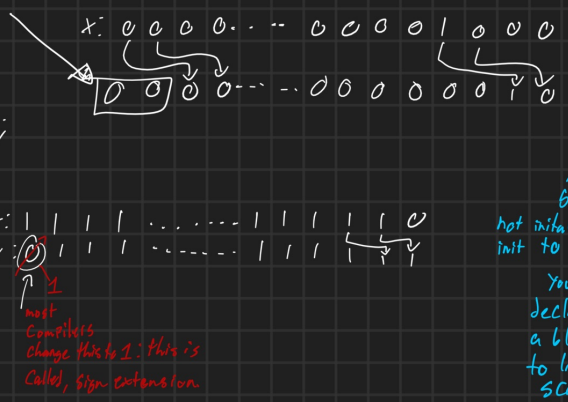
```
int g(int x){
    return x | (1<<5);
}
```

```
int h(int x){
    return x & ~(1<<5);
}
```



Right Shift

```
{
    int x = 8;
    int y = x>>2;
    x = -2;
    y = x>>1;
}
```



Lecture #4 stuff

Storage Classes:

```
int y = 0;
void f(int p){
    printf("%d", p);
    int x;
    x = 0;
    {
        int x;
        x = 1;
        printf("%d", x);
    }
}
```

- global scope
- global variables not initialized will init to 0.
- you can declare a block to limit scope this block defines the scope.
- completely distinct
- (1) Automatic var / local var / stack var
- (2) Static var
  - 2a) global static
  - 2b) file static
  - 2c) Block Static

Logical operations:

```
if(x>3){
if(y){
```

- evaluates if this is true or false
- any pos or neg int → true
- 0 → false

- If first Part of expression in logical AND is false it will not evaluate the right statement. This is called Short circuit evaluation
- logical and
- this follows Pemdas
- || – logical or
- Same for logical OR.

```
if (x>3 && x<15){
```

Assignment:

```
{
    int x, y;
    x = 1;
}
```

- x = 1; has type & value
- assigning it that value
- this is both a expression & statement

```
y = x = 1;
```

- assignment operator goes from right → left
- unlike for example – 5 - 3 - 2 → y = x = 1;

This means you can do

```
x = x + 1;
x += 1;   → x /= 2;
x <<= 1;  x < 1;
```

- Changes x
- does not Change x

Shorthand for adding 1 to a var

```
++x;
```

← Prefix operator

```
Postfix;
int y;
x = 1;
y = x++;
```

- higher Precedence so happens first
- for Postfix: the value before increment is used instead of new value so → y = 1, x = 2

```
f()
```

← Unary function operator

```
-2
```

← unary prefix

```
a[3] <= Postfix;
```

Postfix ++ is used a lot!!

- unary operators

foo2.C

```
extern int x = 5;
void foo2C(){
    x += 1000;
}
```

- fixes this makes it visible to all files.

foo.C

```
int x = 6;
void fooC(){
    x += 100;
}
void barC(){
    x += 200;
}
```

- global var get init on start up & it'll retain the value, updating & changing as the Program Progresses.
- (2a) global static

Linking these results in an error. only 1 copy per global var.

foo4.C

```
int countC(){
    static int n = 0;
    return ++n;
}
```

- limits this only to inside the function but it still keeps & retains its changes in values

foo3.C

```
static int x = 50;
foo3C(){ x += 55; }
bar3C(){ x += 66; }
```

- (2b) file static, var is limited to this file only.
- makes this var only visible in this file